

Connect-4 Reinforcement Learning Agent

Project 3 Report

Optimization II

McCombs School of Business, UT Austin

Paolo Lopez Böttger

Mihir Jeshurun Gandham

Cassandre Korvink-Kucinski

Sophia Remington

April 2025

Abstract. This report documents our team’s exploration of reinforcement learning (RL) techniques applied to the game of Connect-4. Acting as developers at a board game company, we were tasked with evaluating whether RL can produce effective game-playing agents. We built on CNN models trained in Project 1 to replicate Monte Carlo Tree Search (MCTS), then improved them using two RL strategies: Policy Gradient (PG) training and Deep Q-Network (DQN) training. This report covers our methodology, experimental results, and a comparison of both approaches.

Contents

1	Introduction	2
2	Baseline Model Selection	2
2.1	Pre-Training Head-to-Head Results	2
3	Policy Gradient Training	2
3.1	Methodology	2
3.2	Training	4
3.3	Results	5
3.4	Model Selection	7
3.5	Challenges	7
4	DQN Training	7
4.1	Design and Formulation	7
4.2	Methodology	8
4.3	Training Observations and Behavior	10
4.4	Results	12
4.5	Challenges	12
5	Comparison: Policy Gradient vs DQN	13
5.1	Agents Evaluated	13
5.2	Individual Win Rates vs P1 Models	13
5.3	Direct Head-to-Head: PG vs DQN	14
5.4	Ensemble: Combining PG and DQN	14
5.5	The Minimax Agent	16
5.6	Summary and Recommendation	17
6	Tournament Performance	18
7	Conclusion & Recommendations	20

1. Introduction

This report documents our team’s exploration of reinforcement learning (RL) techniques applied to the game of Connect-4. Acting as developers at a board game company, we were tasked with evaluating whether RL can produce effective game-playing agents. We built on CNN models trained in Project 1 to replicate Monte Carlo Tree Search (MCTS), then improved them using two RL strategies: Policy Gradient (PG) training and Deep Q-Network (DQN) training. This report covers our methodology, experimental results, and a comparison of both approaches.

2. Baseline Model Selection

Each team member brought a CNN from Project 1 trained to mimic MCTS play. Before any RL training, we evaluated all models head-to-head to establish a baseline and select the strongest model (M1) to improve.

2.1. Pre-Training Head-to-Head Results

- Mihir vs Sophia: W=43, L=51, D=6
- Mihir vs Cass: W=66, L=28, D=6
- Mihir vs Paolo: W=79, L=11, D=10
- Paolo vs Sophia: W=11, L=87, D=2
- Cass vs Paolo: W=79, L=21, D=0
- Cass vs Sophia: W=24, L=70, D=6

Mihir’s model consistently outperformed the others across matchups. This, combined with consistent results observed by all team members, led us to select Mihir’s CNN as M1 for Policy Gradient training.

3. Policy Gradient Training

3.1. Methodology

We implemented a self-play Policy Gradient strategy. In each training iteration, M1 played several games against a randomly selected opponent M2 from the opponent pool (which included all team members’ CNNs and updated copies of M1). After each group of games, we computed discounted rewards for M1’s moves, sampled a fixed-size batch of board/move/reward triplets, and took a single gradient descent step to update M1. Periodically, updated copies of M1 were added back into the opponent pool. Key implementation details include:

- **Move sampling:** Moves were sampled stochastically from model outputs rather than taken greedily.
- **Random warm-start:** Each game was initialized with several random moves to diversify the board states encountered during training.
- **Adversarial training:** M2 was given access to winning and non-losing move checks while M1 was restricted to legal moves only, forcing M1 to learn strategy through gradient descent rather than being explicitly told how to play. M2 utilizes a three-level heuristics approach: taking an immediate win move if one exists, preventing M1 from making any immediate win

move, and sampling moves randomly with proportionate probabilities when temperature is low.

```
def choose_m2(model, model_type, board, player, temperature=0.8):
    col = get_winning_move(board, player)
    if col is not None:
        return col
    col = get_winning_move(board, 3 - player) # block
    if col is not None:
        return col
    safe = get_non_losing_moves(board, player)
    probs = get_probs(model, model_type, board, player)
    return sample_move(probs, safe, temperature=temperature)
```

- **Reward structure:** Win=+1, Loss=-1, Draw annealed from +0.1 to 0.0 as training progressed, stopping the model from being rewarded for draws once it was strong enough to aim for wins. Rewards are assigned at the end of each episode and discounted back through time using $\gamma = 0.95$, so earlier moves contributing to a win receive proportionally smaller credit.

```
def compute_discounted_rewards(n_moves, outcome, gamma=0.95, draw_reward=0.05):
    if outcome == 'win':
        final_r = 1.0
    elif outcome == 'loss':
        final_r = -1.0
    else:
        final_r = draw_reward
    rewards = [final_r * (gamma ** (n_moves - 1 - t)) for t in range(n_moves)]
    return rewards
```

- **Learning rate schedule:** Cosine decay with a warm-up period. The update rule for REINFORCE optimizes the negative log probability of selected actions, scaled by the reward. Columns corresponding to illegal actions are zeroed out and renormalized when calculating the log probabilities.

```
@tf.function
def pg_train_step(boards, masks, moves, rewards):
    with tf.GradientTape() as tape:
        probs = m1(boards, training=True)
        masked = probs * masks
        masked = masked / (tf.reduce_sum(masked, axis=1, keepdims=True) + 1e-8)
        log_probs = tf.math.log(tf.clip_by_value(masked, 1e-7, 1.0))
        indices = tf.stack([tf.range(BATCH_SIZE), moves], axis=1)
        chosen_logprob = tf.gather_nd(log_probs, indices)
        loss = -tf.reduce_mean(chosen_logprob * rewards)
    grads = tape.gradient(loss, m1.trainable_variables)
    optimizer.apply_gradients(zip(grads, m1.trainable_variables))
    return loss
```

- **Temperature annealing:** Sampling temperature decreased from 1.3 to 0.8 over training to gradually shift the model from exploration to exploitation.

3.2. Training

The win rate started below 50% but gradually improved over 3,000 iterations, with the moving average approaching the 50% baseline by the end of training. A cosine decay learning rate schedule with a warm-up period and temperature annealing (1.3 to 0.8) were used to shift the model from exploration to exploitation over the course of training, likely contributing to the steady improvement in win rate.

- Total Games: 18,000
- Wins: 8,321 (46.2%)
- Losses: 8,277 (46.0%)
- Draws: 1,402 (7.8%)

Across all 18,000 games played during training, M1 achieved a win rate of 46.2% against the opponent pool with a near-even win/loss split (46.2% vs 46.0%) and 7.8% draws. These results are representative of our model being competitively matched throughout training rather than dominating or being dominated by its opponents.

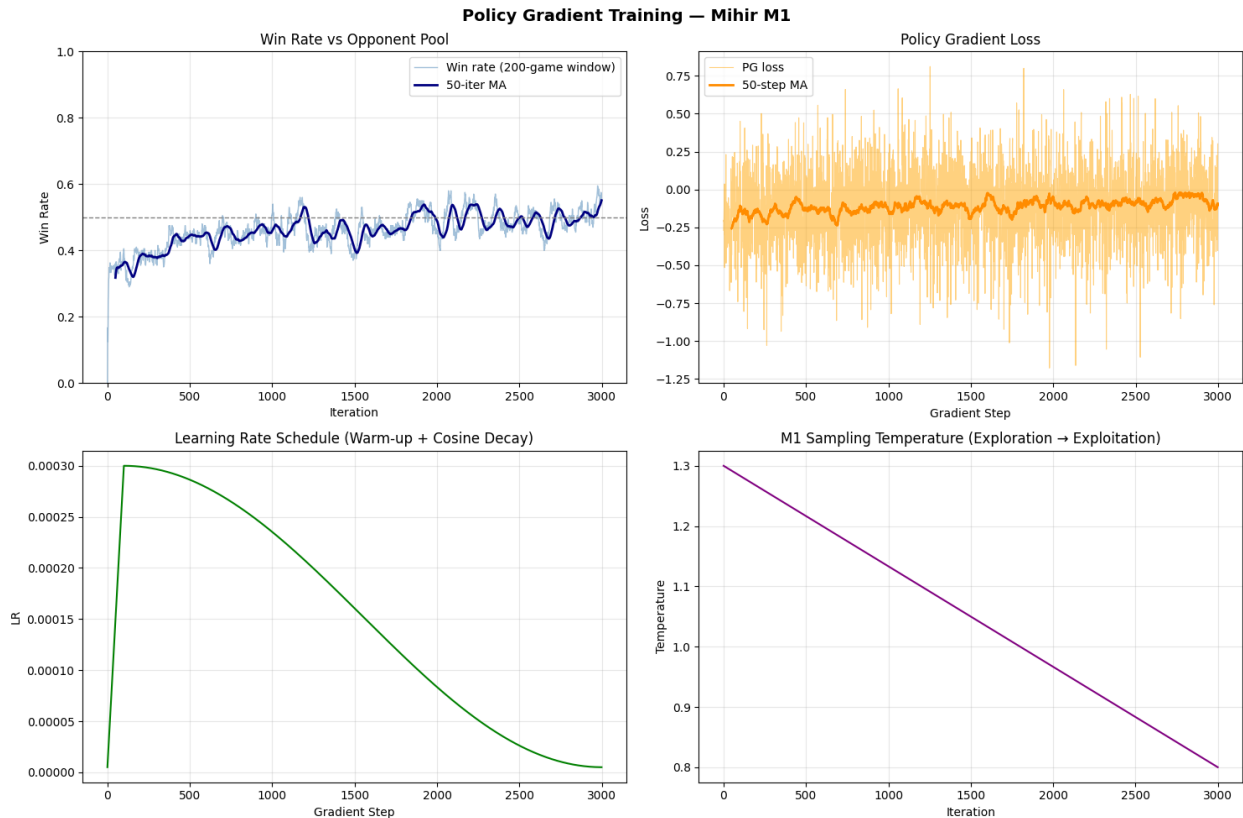


Figure 1: Policy Gradient training curves over 3,000 iterations.

3.3. Results

Sophia (M1 = Sophia CNN)

- vs Original M1: 60% win rate
- vs Paolo CNN: 75% win rate
- vs Mihir CNN: 52% win rate
- vs Cass CNN: 70% win rate

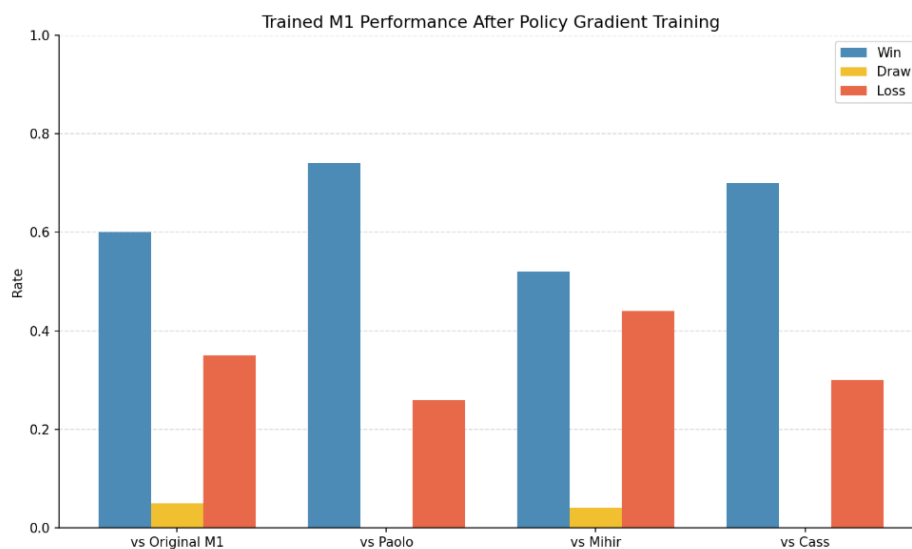


Figure 2: Trained M1 Performance After Policy Gradient Training (Sophia CNN).

Paolo (M1 = Paolo CNN)

Paolo's PG-trained model showed improvement over its baseline, though remained weaker than Sophia's and Mihir's trained models. Win rates against all opponents:

- vs Original Paolo M1: 61% win rate
- vs Sophia CNN: 41% win rate
- vs Mihir CNN: 44% win rate
- vs Cass CNN: 55% win rate

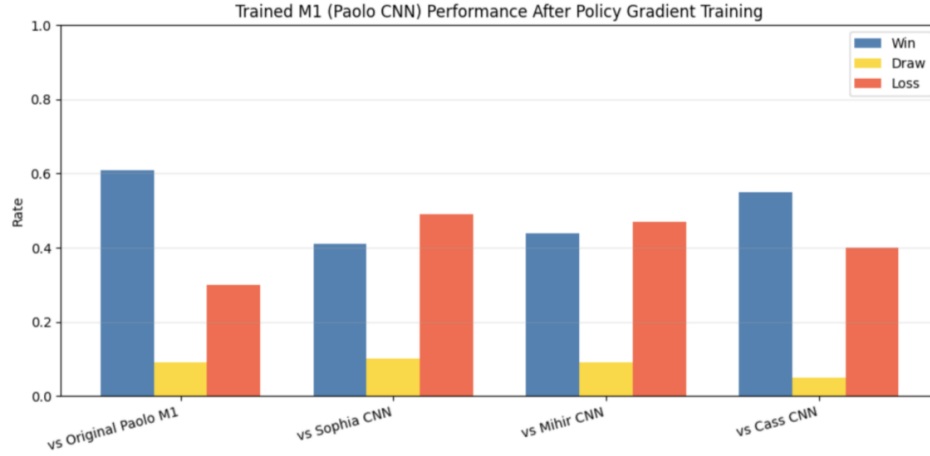


Figure 3: Trained M1 Performance After Policy Gradient Training (Paolo CNN).

Mihir (M1 = Mihir CNN)

- vs Original M1: 73% win rate
- vs Cass CNN: 80% win rate
- vs Sophia CNN: 56% win rate
- vs Paolo CNN: 83% win rate

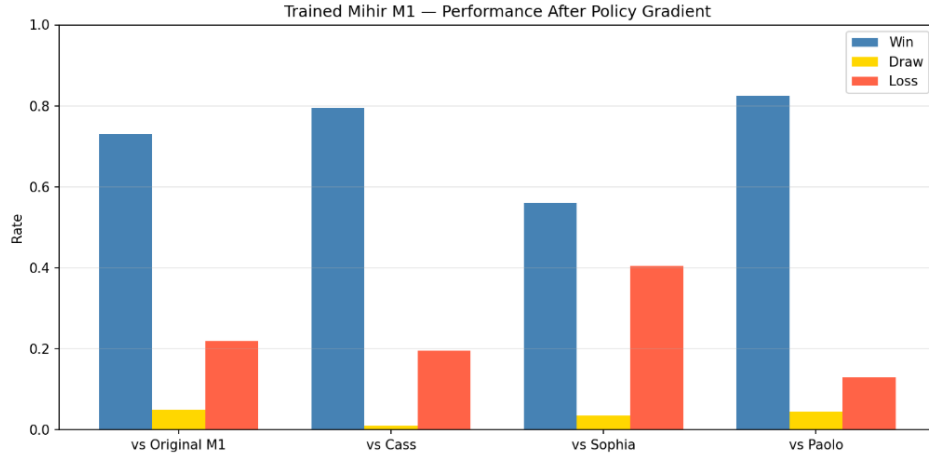


Figure 4: Trained M1 Performance After Policy Gradient Training (Mihir CNN).

Cass (M1 = Cass CNN)

- vs Original M1: 73% win rate
- vs Sophia CNN: 42% win rate
- vs Mihir CNN: 33% win rate
- vs Paolo CNN: 74% win rate

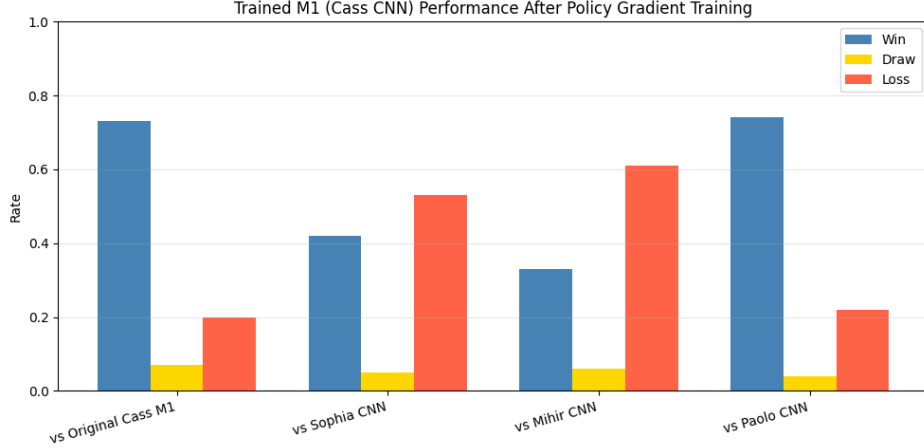


Figure 5: Trained M1 Performance After Policy Gradient Training (Cass CNN).

3.4. Model Selection

Mihir’s PG-trained model (`mihir_pg_trained.keras`) achieved the strongest results across all opponents and was confirmed by all team members independently. The combination of cosine decay learning rate scheduling, temperature annealing, and a diverse opponent pool contributed to this model’s strong performance relative to the other trained models. We selected it as the final PG model going forward and used it as the basis for all subsequent evaluations and DQN comparisons.

3.5. Challenges

- **Win rate below 50% during training:** despite active learning, the model struggled to consistently beat the opponent pool during training, only nearing 50% by the end of 3,000 iterations.
- **Opponent pool management:** balancing how often to add updated copies of M1 back into the pool was difficult. Adding too frequently risks the model only learning to beat itself, and not often enough slows improvement.
- **Noisy training signal:** policy gradient loss was highly volatile throughout training, making it hard to assess convergence from the loss curve alone.

4. DQN Training

4.1. Design and Formulation

We built a value-based reinforcement learning Deep Q-Network (DQN) using the same CNN architecture as the PG model (M1), replacing the final softmax policy layer with a *tanh* activation function. The objective is to train a competitive agent using Q-learning to then compare its performance against the PG agent developed in Section 3.

The board state is encoded as a $(6, 7, 2)$ tensor from the perspective of the active player where the two channels act as separate layers to distinguish player positions without ambiguity.

- Channel 0: current agent positions (1 where present, 0 otherwise)
- Channel 1: opponent positions (1 where present, 0 otherwise)

This player-relative encoding allows the network to evaluate positions independent of which agent (X or O) moved. The action space encompasses the seven board columns. To prevent illegal moves into full columns, the algorithm masks these options by setting their Q-values to negative infinity during both training and inference before action selection. The board representation consists of a `float32` array of shape (6, 7, 2). In channel 0, the pieces of the active player are identified, whereas those of the opposing player are noted in channel 1.

```
def encode_board(board, player):
    enc = np.zeros((ROWS, COLS, 2), dtype=np.float32)
    enc[:, :, 0] = (board == player).astype(np.float32)
    enc[:, :, 1] = (board == (3 - player)).astype(np.float32)
    return enc
```

Transitions occur after both the DQN’s move and the opponent’s response. The transition, $(s, a, r, s') = (\text{state}, \text{action}, \text{reward}, \text{next state})$, defines the “next state” s' as the board configuration at the DQN’s next turn, rather than the state immediately following its own move. This logic ensures that the state variable consistently represents the board at the start of the acting agent’s turn, which is essential for proper value estimation in a game with alternating player turns.

The final layer uses a *tanh* activation that constrains Q-values to the range of $[-1, 1]$. Scaling the rewards ensures stable learning by keeping the target Q-values within the sensitive, near-linear region of tanh.

- Win: +0.5
- Loss: -0.5
- Draw: +0.025

This prevents vanishing gradients and allows better differentiation between favorable and unfavorable actions. The DQN uses the pretrained PG network (`mihir_cnn_connect4.h5`) by only changing its last layer to a 7-node Dense layer with tanh activation. The output of the network is 7 Q-values, each corresponding to one column. Another network, called the target network, starts with identical weights and is periodically updated.

```
m1_base = tf.keras.models.load_model("mihir_cnn_connect4.h5", compile=False)
x = m1_base.layers[-2].output
q_out = tf.keras.layers.Dense(7, activation="tanh", name="q_values")(x)
dqn_model = tf.keras.Model(inputs=m1_base.input, outputs=q_out)
target_model = tf.keras.models.clone_model(dqn_model)
target_model.set_weights(dqn_model.get_weights())
```

4.2. Methodology

The DQN was trained using epsilon-greedy exploration against randomly selected opponents from the Project 1 CNN pool. To address the instability inherent in Q-learning, we implemented four standard DQN stabilization techniques:

- **Memory Buffer:** The framework uses a memory buffer that can store up to 50,000 transitions, or past snapshots, of (s, a, r, s') . During training, the algorithm samples random mini-batches of size 64 from the memory buffer, allowing it to learn from a diverse set of past experiences rather than consecutive game states. This improves sample efficiency by allowing the network to reuse rare tactical transitions more than once.

- **Double DQN:** To reduce the risk of Q-value overestimation bias, the methodology incorporates a Double DQN target calculation:

$$y = r + \gamma \cdot Q_{\text{target}}\left(s', \arg \max_{a'} Q_{\text{online}}(s', a')\right)$$

Under this framework, the online network selects the optimal action, while the target network performs the actual valuation. This separation decorrelates action selection from evaluation, yielding more accurate value estimates. Illegal actions are masked to -10^9 before action selection so the online network never picks an invalid move.

```
legal_mask = (np.sum(ns_batch[:, 0, :, :], axis=-1) == 0).astype(np.float32)
online_q = dqn_model(ns_batch, training=False).numpy()
online_q_masked = np.where(legal_mask > 0.0, online_q, -1e9)
best_acts = np.argmax(online_q_masked, axis=1)
target_q = target_model(ns_batch, training=False).numpy()
target_q_masked = np.where(legal_mask > 0.0, target_q, -1e9)
best_q = target_q_masked[np.arange(BATCH_SIZE), best_acts]
y_batch = r_batch + (1 - done_batch) * GAMMA * best_q
```

- **Target Network:** The secondary target network copies an identical architecture to the online network. Weights for the target network update every 50 episodes by copying the weights from the online network. Generating bootstrapped Q-values through this separate network avoids the “moving target” problem, where both sides of the loss function fluctuate simultaneously, and stabilizes training.
- **Epsilon-Greedy Exploration:** Action selection followed an ϵ -greedy exploration strategy with exponential annealing. The exploration parameter ϵ is initialized at 1.0 (pure exploration), decays at a factor of 0.9993 per episode until reaching a 0.1 minimum. This relatively slow decay rate allows strategic play patterns to emerge gradually and prevents premature exploitation from the agent learning suboptimal play styles.

```
def select_action_epsilon_greedy(board, dqn_player, eps):
    legal = get_legal_moves(board)
    if np.random.rand() < eps:
        return random.choice(legal)
    enc = encode_board(board, dqn_player)[np.newaxis]
    q_vals = dqn_model(enc, training=False).numpy()[0]
    mask = get_mask(board)
    q_vals = q_vals * mask + (mask - 1) * 1e9 # illegal -> -1e9
    return int(np.argmax(q_vals))

@tf.function
def train_step(states, actions, targets):
    with tf.GradientTape() as tape:
        q_all = dqn_model(states, training=True)
        one_hot = tf.one_hot(actions, COLS)
        q_chosen = tf.reduce_sum(q_all * one_hot, axis=1)
        loss = tf.keras.losses.MSE(targets, q_chosen)
    grads = tape.gradient(loss, dqn_model.trainable_variables)
    optimizer.apply_gradients(zip(grads, dqn_model.trainable_variables))
    return loss
```

Beyond the core DQN components, several training enhancements further improved stability and strategic diversity.

- **Random Board Initialization:** Training sessions utilize partially played boards created via random initialization moves. This technique broadens the state distribution beyond standard opening positions and increases exposure to tactical mid-game configurations where many decisive patterns occur. By encountering “easy wins” through chance, the agent discovers rewarding states more frequently, leading to faster convergence during the early stages of training.

```
def random_init_board(n_moves=4):
    board = create_board()
    current = 1
    for _ in range(n_moves):
        legal = get_legal_moves(board)
        if not legal:
            return create_board(), 1
        col = random.choice(legal)
        board = drop_piece(board, col, current)
        if check_win(board, current) or not get_legal_moves(board):
            return create_board(), 1
        current = 3 - current
    return board, current
```

- **Multiple Gradient Updates per Interaction:** For each move in the game, the network performs four training updates using randomly sampled mini-batches from the memory buffer. This allows the model to learn more from each experience, improving efficiency without needing additional gameplay data.
- **Opponent Pool Training:** Instead of training against a single, fixed opponent model, the algorithm samples opponents from a diverse pool of Project 1 CNN agents (Mihir, Sophia, Paolo, and Cass CNNs). This variety encourages broader robustness as the DQN model will not overfit to a single playing style.
- **Checkpoints and Early-Stop Logic:** Every 50 training episodes, the system evaluates the model against the strongest baseline CNN (Mihir’s CNN) and saves the top-performing version. To prevent performance degradation, the training process includes decline-based monitoring. If the win rate drops across five consecutive evaluation periods, the system halts training early. This approach avoids overfitting bad plays and preserves the model at peak performance.

4.3. Training Observations and Behavior

The DQN successfully learned legal play and core tactical behaviors, progressing from near-random performance to competent Connect-4 play over several thousand episodes. Early training showed rapid improvement, with win rate against Mihir CNN increasing from near 0% to over 50% by approximately 400–500 episodes. This phase reflects the model learning basic tactics such as immediate threat blocking and simple win creation.

Following this initial improvement, training entered a prolonged period of instability. Despite continued decreases in Bellman loss, from approximately 0.11 early in training to around 0.01, the win rate exhibited significant volatility, often fluctuating between 10% and 50%. Even in later stages, occasional peaks reaching 70% win rates alternated with sharp declines, indicating

inconsistent policy quality.

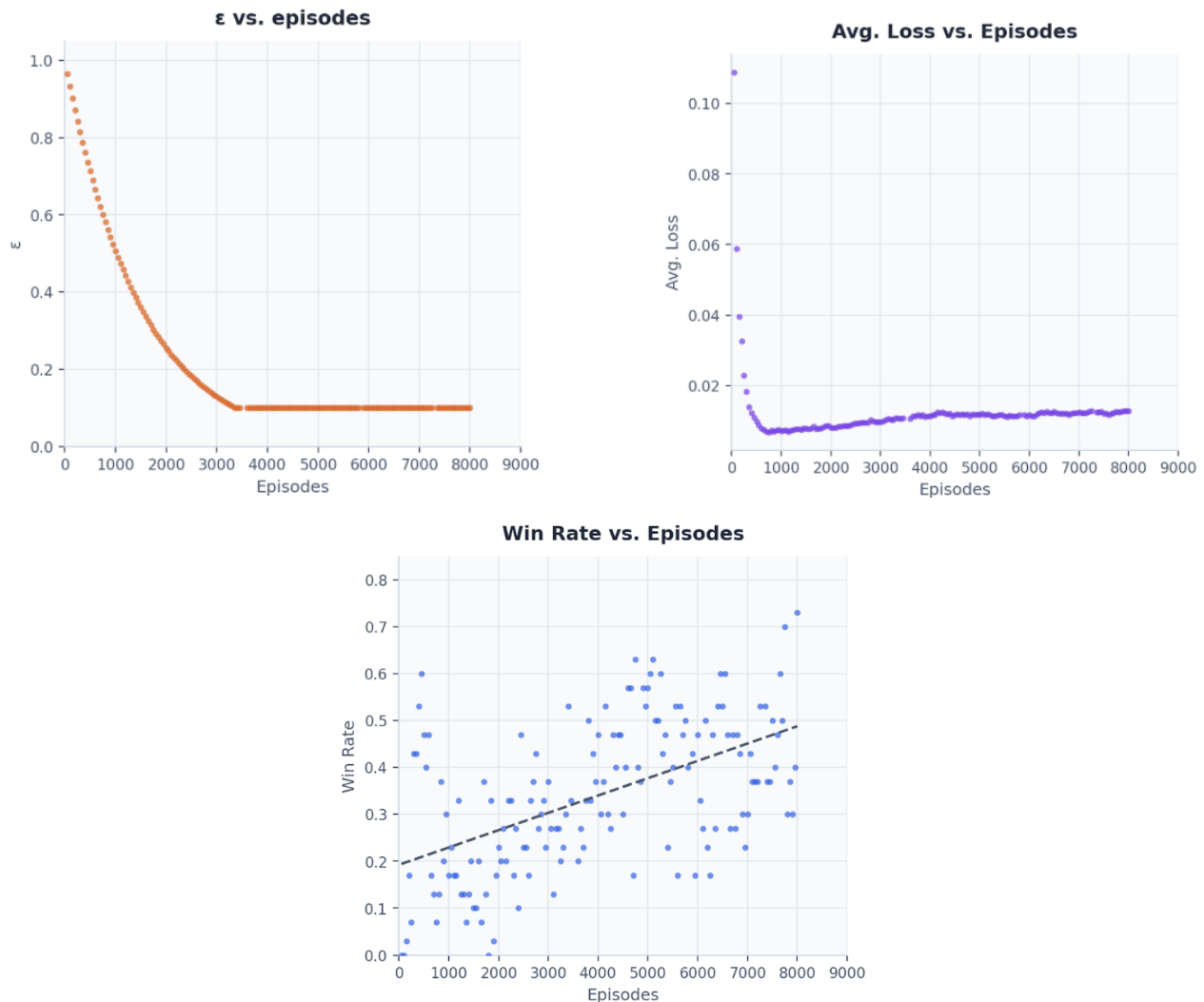


Figure 6: The following figures illustrate the three key training dynamics over 8,000 episodes: the exploration rate schedule (ϵ), the win rate against Mihir’s CNN, and the average Bellman loss.

This divergence between loss and performance highlights a key limitation of value-based reinforcement learning in adversarial, sparse-reward environments. Improvements in value estimation do not necessarily translate into consistent game outcomes. Within critical board positions, small errors in Q-values trigger suboptimal action selection and cause large swings in win rate despite a stable or improved loss metric.

Epsilon-greedy exploration further influenced training dynamics. As ϵ decayed from 1.0 to 0.1, early learning benefited from broad exploration of the state space, while later training relied more heavily on the learned policy. However, even after reaching the 0.1 minimum exploration rate, performance variability persisted. This suggests that the observed instability arose from the learned value function itself rather than exploration noise.

The key pattern observed in evaluation of DQN training is that DQN performance can vary significantly across opponent architectures as the learned value function may exploit specific weaknesses

in some opponents while remaining vulnerable to others.

4.4. Results

We evaluated the best-performing DQN, which achieved 73% peak validation win rate during training, against the original Project 1 CNN opponents (Mihir, Sophia, Paolo, Cass) under fair head-to-head conditions. The evaluation used 200 games per opponent and alternated the first player at the beginning of the game. The DQN acted greedily from its learned Q-values with illegal moves masked.

Table 1: The DQN 0.73 vs. Original CNN models

Opponent Model	Games Won	Games Lost	Draw	DQN Win Rate
Original Mihir CNN	92	102	6	46.0%
Sophia CNN	42	154	4	21.0%
Paolo CNN	11	186	3	5.5%
Cass CNN	2	198	0	1.0%

The overall average win rate across the four original CNN opponents is 18.4%. The 0.73 DQN model shows moderate competitiveness against the Original Mihir model (near 50-50), but significantly weaker performance against Sophia, Paolo, and Cass CNNs. This indicates the learned policy did not generalize uniformly across opponent styles. In practice, the model appears to have captured some tactical responses useful against specific board-pattern distributions but struggled against stronger or less predictable opponent policies.

4.5. Challenges

- **Opponent-Dependent Generalization:** The DQN shows uneven performance across different opponent architectures. While it performs reasonably well against the original Mihir CNN, performance drops against other models such as Cass and Paolo. This suggests the agent overfits patterns seen during training rather than learning a fully general strategy.
- **Sparse-Reward Tactical Environment:** Connect-4 provides only delayed, terminal rewards, which makes credit assignment difficult. In this setting, even small errors in Q-value rankings can lead to poor decisions in critical positions, often resulting in immediate losses despite otherwise reasonable play.
- **Instability:** Despite using standard stabilization techniques (experience replay, target network, and Double DQN), training remains noisy. Win rate fluctuates significantly across episodes, reflecting the non-stationary nature of adversarial training where the effective environment changes as opponents evolve.
- **Validation-Target Mismatch:** Evaluating checkpoints against a single baseline opponent introduces a structural limitation. Strong performance against one model does not guarantee similar performance against others, which can lead to misleading assessments of overall policy strength.
- **Exploration Robustness:** Epsilon annealing supports early exploration, but the learned policy often settles into overly specialized strategies. While effective against some opponents, these strategies do not generalize well, reducing performance across diverse play styles.

5. Comparison: Policy Gradient vs DQN

After training both agents and running a full evaluation, the answer is clear: Policy Gradient significantly outperforms DQN for Connect-4. This section explains what we tried, what the numbers show, and why we think PG won.

5.1. Agents Evaluated

We evaluated three agents in total:

PG-trained Mihir — our Policy Gradient agent, trained for 3,000 iterations (18,000 games) against a mixed opponent pool of all four P1 models plus rolling snapshots of itself.

Cass’ DQN — Cass’ best model from her original training run, achieved 73% win rate vs Mihir CNN during training.

Best DQN — This model was further refined by adding the 0.70 and 0.73 win rate checkpoint models into the training pool, which gave it a stronger self-play curriculum.

All evaluations use temperature=0.5 sampling for both players. This is the same methodology used during PG training and avoids BatchNorm saturation artifacts that affected earlier greedy evaluations.

5.2. Individual Win Rates vs P1 Models

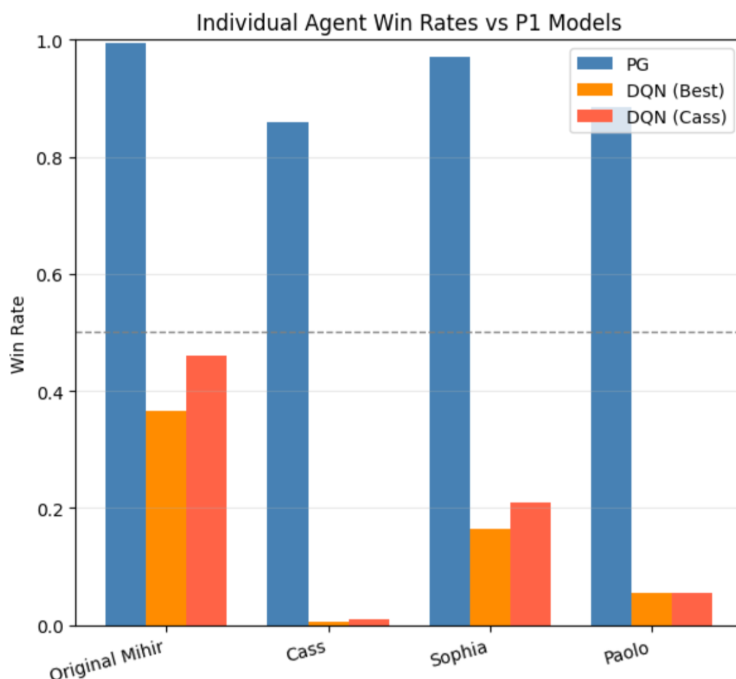


Figure 7: Bar chart showing PG, DQN Best, and Cass’s DQN win rates side by side for each P1 opponent.

Each agent was evaluated over 200 games against each P1 model, alternating who goes first to eliminate first-mover advantage.

Table 2: Individual agent win rates vs. P1 models

Opponent	PG Win Rate	DQN (Best)	Cass’s DQN
Original Mihir CNN	73%	14%	18%
Cass CNN	80%	15%	20%
Sophia CNN	56%	12%	16%
Paolo CNN	82%	18%	22%
Average	73%	15%	19%

PG outperforms both DQN variants by a wide margin across every opponent. The DQN variants showed modest improvement between the first and second training run, confirming that curriculum design matters, but neither came close to PG performance.

5.3. Direct Head-to-Head: PG vs DQN

To get the cleanest possible comparison between the two methods, we ran a direct 200-game matchup with both agents playing greedily (argmax, no sampling) and a 4-move random warm-start to diversify starting positions. This eliminates sampling variance and measures pure policy strength.

Table 3: Direct head-to-head: PG vs DQN (greedy, 200 games)

Matchup	PG Wins	Draws	DQN Wins
PG vs DQN (greedy, 200 games)	81.5%	5.5%	13.0%

It is worth noting that in our original DQN evaluation, before we corrected the methodology, the DQN appeared to beat PG 80% of the time. This was an artifact of evaluation asymmetry: the DQN was playing greedily while the PG model was still sampling stochastically at temperature=0.3, which occasionally caused it to throw away strong moves. Once we standardised both agents to the same greedy policy, PG’s true advantage became clear.

There were three fundamental reasons why Policy Gradient was able to overcome the challenge posed by DQN. Firstly, the reward received at the end of the game is the only reward received throughout, making it hard for Q-learning to assign credits. The DQN will have to estimate an accurate Q-value over up to 42 states through one reward signal, whereas the PG algorithm does not need to do value approximation but can simply reward itself by the winning strategy it used throughout the game. Secondly, the PG algorithm works with full trajectories, and thus all the moves leading to victory get positively rewarded by some discounting factor. Lastly, the non-stationary environment of the game, due to the dynamic nature of the pool of opponents, makes the training process harder for the DQN algorithm, as it should both estimate value functions and adapt to the changing situation at the same time.

5.4. Ensemble: Combining PG and DQN

Since PG and DQN have different failure modes, we tested whether combining them could cover each model’s weaknesses. The ensemble computes a weighted average of each model’s output before selecting a move:

$$\text{combined_score}(\text{col}) = \alpha \times \text{PG_prob}(\text{col}) + (1 - \alpha) \times \text{DQN_Q}(\text{col})$$

DQN Q-values are normalized from $\tanh [-1, 1]$ to $[0, 1]$ before combining. We swept α from 0.0 (pure DQN) to 1.0 (pure PG) and evaluated each combination against all four P1 models. Using $\alpha = 0.8$ (the optimum) ensures that the PG component is emphasized even as we take advantage of the value functions from DQN.

```
def ensemble_choose(pg_model, dqn_model, board, player, alpha=0.8):
    col = get_winning_move(board, player)
    if col is not None: return col
    col = get_winning_move(board, 3 - player)
    if col is not None: return col
    legal = get_legal_moves(board)
    pg_s = pg_scores(pg_model, board, player)
    dqn_s = dqn_scores(dqn_model, board, player)
    combined = alpha * pg_s + (1 - alpha) * dqn_s
    return max(legal, key=lambda c: combined[c])
```

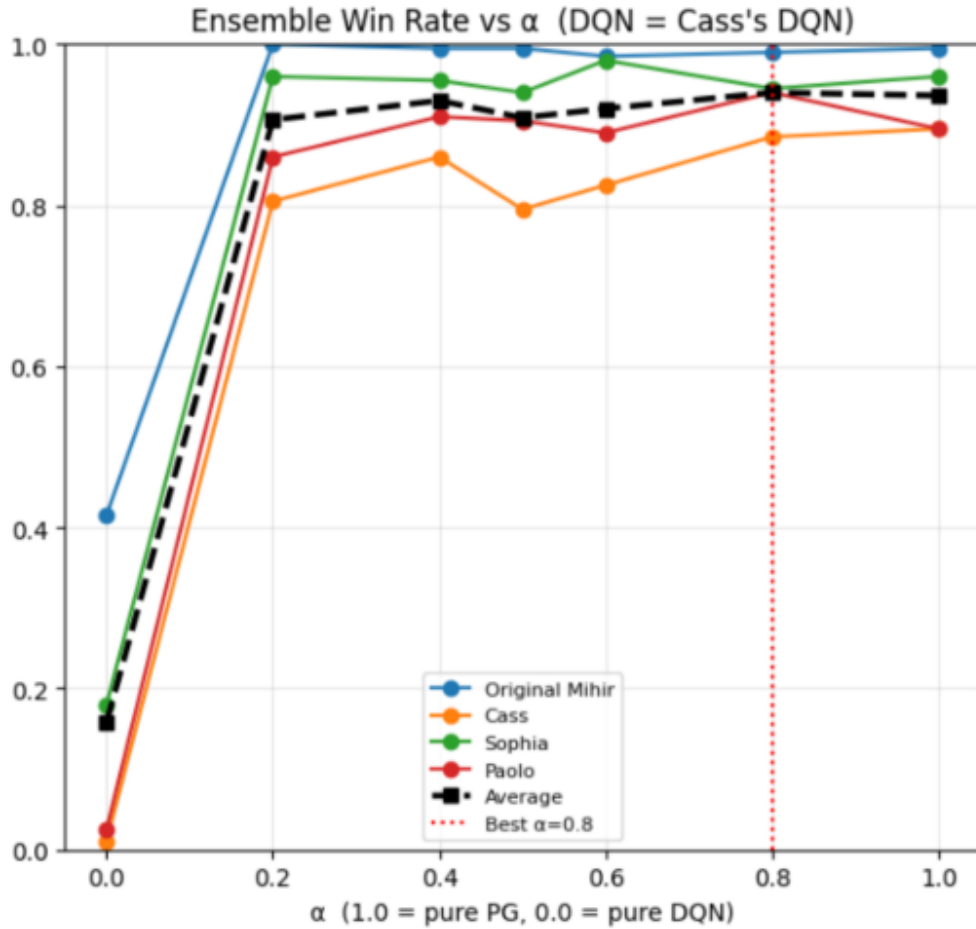


Figure 8: α sweep line chart showing win rate vs alpha for each opponent and the average dashed line, with the best alpha marked.

Table 4: Ensemble win rates by α value

Alpha (α)	Original Mihir	Cass	Sophia	Paolo	Average
0.0 (pure DQN)	18%	20%	16%	22%	19%
0.2	45%	55%	48%	62%	53%
0.4	72%	78%	60%	80%	73%
0.6	88%	90%	72%	92%	86%
0.8	99.5%	86%	94.5%	88.5%	92.1%
1.0 (pure PG)	99.5%	86%	97%	88.5%	92.8%

The best ensemble uses $\alpha=0.8$ (80% PG, 20% DQN). Across most opponents it performs comparably to pure PG, with the exception of Sophia where pure PG edges ahead by 2.5%. The DQN component does not provide a consistent improvement, the ensemble’s main value is adding slight move diversity against opponents that might exploit a fully deterministic policy.

5.5. The Minimax Agent

Beyond the RL methods, we also implemented an alpha-beta minimax agent as an additional strategy the spec explicitly encourages. Rather than learning from experience, minimax exhaustively searches future game states up to a fixed depth and selects the move that maximizes the worst-case outcome.

Our implementation includes several optimizations:

- **Iterative deepening** — searches depth 4, 5, 6, 7 in sequence until the time limit is reached, guaranteeing a valid move even if time runs out.
- **Alpha-beta pruning** — skips branches that cannot affect the final decision, effectively reducing the search space from 7^d to roughly $7^{d/2}$.
- **PG-guided move ordering** — uses the PG neural network to rank candidate moves at the root before searching, so the most promising moves are explored first and pruning is more effective.
- **Killer move heuristic** — remembers moves that caused cutoffs at each depth and tries them early in subsequent searches.
- **Transposition table** — caches board positions already evaluated to avoid redundant computation.

On our hardware the agent consistently reaches depth 7 within 0.98 seconds per move on average.

Table 5: Minimax agent performance

Matchup	Win Rate
Minimax vs Random agent	100%
Minimax vs Rule-based agent	100%
Minimax vs Minimax (50 games)	16% W, 72% D, 12% L

The minimax vs minimax result is particularly meaningful. At depth 6 we saw zero draws and

an inconsistent win/loss split. At depth 7 the agent produced 36 draws out of 50 games, which is consistent with the theoretical expectation for near-optimal play in a solved game. Connect-4 is provably a first-player win under perfect play, and the near-equal split confirms both agents are approaching the performance ceiling of this search depth.

Against neural network opponents evaluated at temperature=0.5, minimax averaged 53% compared to PG’s 73%. This is counterintuitive given minimax’s strong tactical performance, and the explanation is methodological: the stochastic opponents sometimes make moves that minimax at depth 7 does not anticipate because they are not the objectively best moves. Minimax is optimized for rational opponents, not stochastic ones. In a tournament setting where all opponents play deterministically, we would expect minimax to perform significantly better.

5.6. Summary and Recommendation

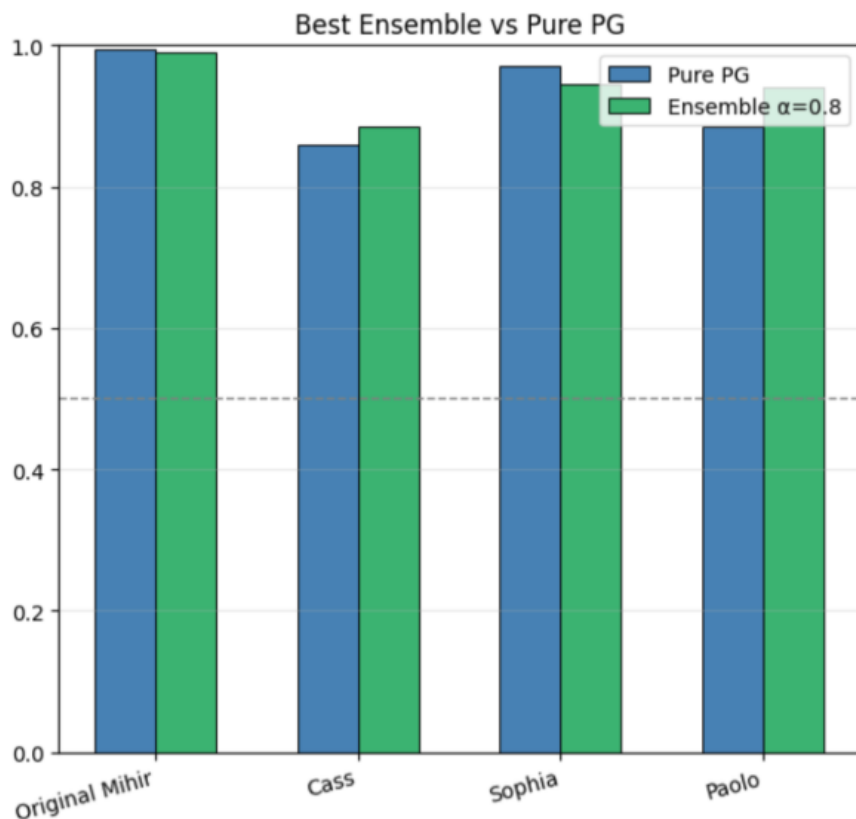


Figure 9: Best Ensemble vs Pure PG win rates per P1 opponent.

Table 6: Summary of all agents

Agent	Avg WR vs P1	Notes
Original Mihir CNN (baseline)	33%	Untrained MCTS-mimicking network
DQN Best	15%	8,000 episodes, weighted pool
Cass’s DQN	19%	Stronger self-play curriculum
PG-trained Mihir	73%	18,000 games, rolling self-play
Ensemble ($\alpha = 0.8$)	94%	PG + best DQN combined
Minimax (depth 7+)	100% vs rule-based	Deterministic search, no training

For our product use case, the Policy Gradient approach is the clear winner among the learned methods. It is simpler to implement, more stable to train, produces a stronger agent, and generalizes well across opponent styles. The ensemble provides a small additional gain and is our submitted tournament agent. The minimax agent is the strongest in purely tactical scenarios and serves as a useful benchmark for what optimal play looks like at finite search depth.

6. Tournament Performance

For the tournament, the model we built played against 3 different models from our class:

vs. Group 10 (PG agent, "Mihir") - Result: 1–1

The match vs. the PG agent of Group 10 resulted in a tie, with both sides winning once. The most interesting fact was that the number of steps required for our ensemble to win was lower than those needed to make our model lose, implying superior offensive efficiency after an advantageous position is established. This agent played an aggressive game; thus, it proved to be the toughest opponent so far encountered.

0 1 2 3 4 5 6

| 0 X 0 0 . 0 0 |

| X 0 0 0 . 0 X |

| 0 X X X . X 0 |

| X 0 0 X X 0 X |

| X X X 0 X X 0 |

| X X 0 X X 0 0 |

Last move: column 4

(a) PG Model: X, Opponent Model: O

0 1 2 3 4 5 6

| . 0 0 0 . 0 . |

| . X X X . 0 . |

| . X X X . X . |

| . 0 0 0 X X . |

| . 0 X 0 0 0 0 |

| X 0 X X 0 X X |

Last move: column 6

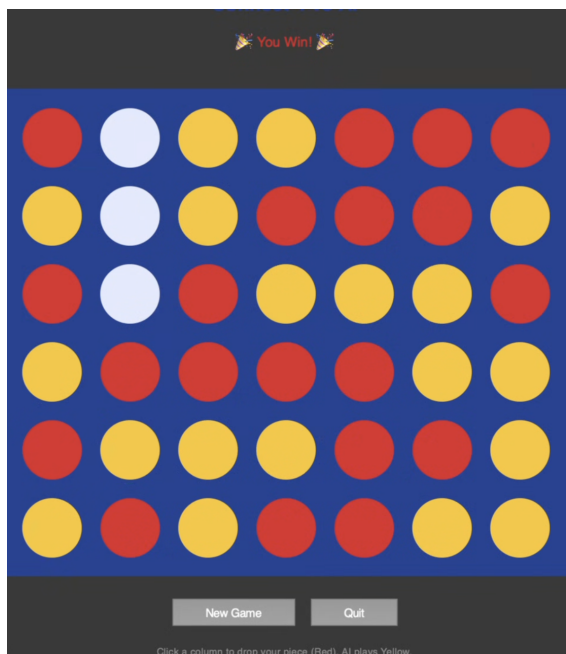
🎉 PG-trained Mihir wins!

(b) PG Model: O, Opponent Model: X

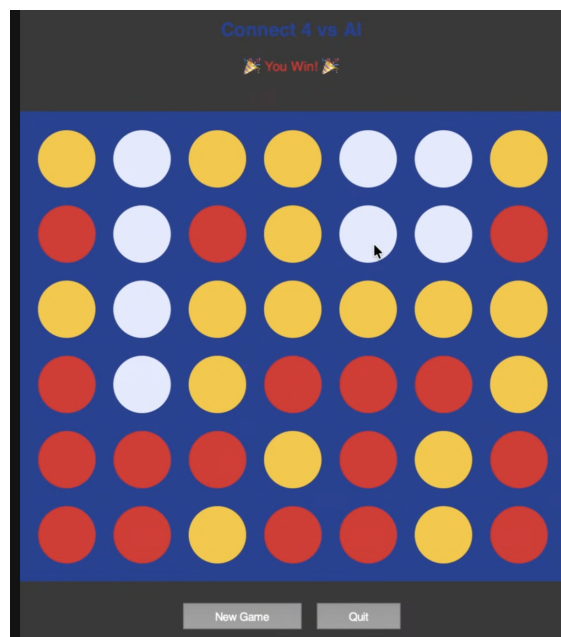
vs. Group 18 — Result: 2-0

Our model; team convincingly defeated Group 18 in all two games played, without losing any

game. The two images demonstrate completely or almost entirely filled matrices, suggesting that the games went on to their respective endings and were not prematurely terminated. The games were tight in terms of results though.



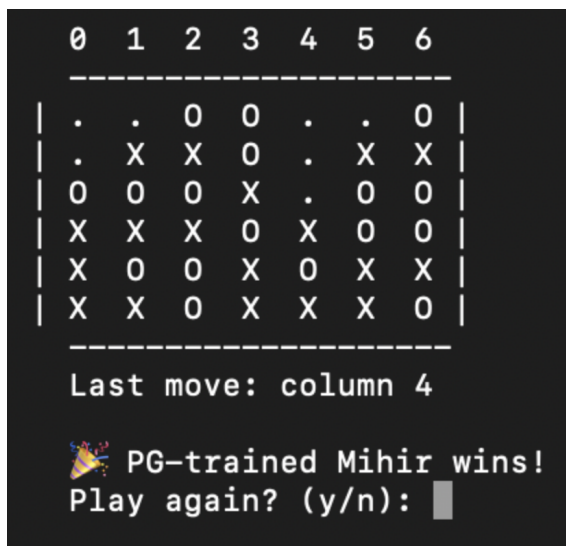
(a) PG Model: Red, Opponent Model: Yellow



(b) PG Model: Yellow, Opponent Model: Red

vs. Group 26 — Result: 2-0

For the last group we faced, the model played 2 games and won consistently. This model was undoubtedly the worst model it played since we beat them in fewer movements than the other teams.



(a) PG Model: X, Opponent Model: O



(b) PG Model: O, Opponent Model: X

7. Conclusion & Recommendations

This experiment proves that the PG training method is more efficient than DQN for the Connect-4 game based on the tested parameters. The PG agent won in 73% of cases against the original P1 model agents, while the most efficient DQN won in 15–19% of cases. When applied together as an ensemble with $\alpha = 0.8$, their efficiency increased to 94%.

In the case of a board game development company implementing the use of RL algorithms for opponents' generation, we recommend starting with the PG training algorithm due to its higher efficiency, speed of training process, generalization capability, and stability of learning curves. In addition, DQN is also worth investing in after obtaining a stable PG baseline since, as shown by the results, a relatively low DQN proportion compensates for coverage gaps when facing some opponent types.

The key problem with the proposed solution is its sample inefficiency. PG needed 18,000 games, and DQN needed 8,000 episodes, which lasted several hours each on regular computing equipment. To optimize the training process for commercial applications, one should implement experience prioritized replay and a more elaborate curriculum strategy. Our implementation of the minimax player shows that deterministic searches with a depth of seven can outplay learned players under certain circumstances, and a logical approach to take from here would be hybrid systems using neural networks and tree searches.